MINI PROJECT REPORT

KAISEI CLIENT DESK

Submitted by:
GVS Prudhvi Raj
2511021060085

In partial fulfilment for the award of the degree of
BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING
(Kalvium Work-Integrated Learning Program)

Under the guidance of
Vaibhav Giri
Associate Professor, ASAC

ALLIANCE SCHOOL OF ADVANCE COMPUTING
ALLIANCE UNIVERSITY, BENGALURU
October-2026

COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Kaisei Client Desk** is submitted by **GVS Prudhvi Raj** [Uni. Roll No. **2511021060085**] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026–2027.

This report embodies the results of original work and studies conducted by students and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

Vaibhav Giri
Associate Professor
Alliance School of Advanced Computing

COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **KAISEI CLIENT DESK** using Node.js, Express, PostgreSQL, Prisma ORM, and React, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of **Vaibhav Giri**, Associate Professor.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

GVS Prudhvi Raj 2511021060085

Name of the Student University Registration Number Signature

Full-Stack Development Report — Kaisei Client Desk

Oct 26, 2026 · @GVS Prudhvi Raj

This report documents a small working full-stack web application - a React frontend, a Node.js/Express REST API and a PostgreSQL database - through Kaisei Client Desk, a client, project, task and invoice tracker for a freelance web studio.

1. The Problem the Application Solves

A freelance web studio juggles many clients at once. Every client project breaks down into small work items - "design the booking calendar", "compress hero images", "test the UPI flow" - and every finished milestone becomes an invoice that somebody must remember to collect. Notebooks and chat threads lose these things. Kaisei Client Desk keeps all of it in one relational database behind one browser interface: **clients**, their **projects**, the **studio tasks** inside each project, and the **invoices** raised against each project. A dashboard shows the money position (billed, collected, outstanding) at a glance.

2. The CRUD-over-REST Concept

Every record type in the app supports the four CRUD operations - Create, Read, Update, Delete - and each operation is exposed as a REST endpoint: URLs name resources (nouns such as `/api/tasks/7`) and HTTP methods express the action. The mapping used throughout the project:

CRUD operation	HTTP method	SQL it becomes	Example in this project
Create	POST	INSERT	`POST /api/tasks` adds a studio task (201)
Read	GET	SELECT	`GET /api/tasks?projectId=3` lists a project's tasks (200)
Update	PUT / PATCH	UPDATE	`PUT /api/tasks/7` edits it; `PATCH /api/tasks/7/toggle` flips done (200)
Delete	DELETE	DELETE	`DELETE /api/tasks/7` removes it (200; a missing id answers 404)

Responses are JSON, and the status code always says what happened: 200 OK, 201 Created, 400 validation failure, 404 missing resource, 409 business conflict (duplicate client email, paying an invoice twice, deleting a client that still has projects), 503 database unreachable.

3. Technologies Used

Technology	Role in this project
React 19 + Vite	component-based frontend and dev server (localhost:5173)

HTML, CSS, JavaScript (ES modules, async/await)	page structure, hand-written styling, one language across both ends
Node.js + Express 5	REST API server (localhost:5000)
PostgreSQL	persistent relational database - data survives server restarts
Prisma 7	ORM: schema, client generation, migrations, parameterised queries
Postman	API testing (31 saved requests including failure cases)
Git / GitHub	version control and project hosting

4. Data Model and Persistence

The schema is four related tables. A task cannot exist without its project, and a project cannot exist without its client, so foreign keys enforce the chain; the migration that creates the tables is versioned with the code, and a seed script (`npm run seed`) recreates the demo dataset on any machine:

```
clients (id PK, name, company, email UNIQUE, phone, ...)
   |<-- projects (id PK, clientId FK, title, brief, deadline,
   |              status DRAFT/ACTIVE/DELIVERED/CANCELLED, budgetPaise, ...)
   |      |<-- tasks (id PK, projectId FK, title, notes, dueDate, done, ...)
   |      |<-- invoices (id PK, projectId FK, amountPaise,
   |                     status UNPAID/PAID/VOID, issuedOn, dueOn, paidOn, ...)
```

Because all records live in PostgreSQL, nothing is lost when the backend stops: the server was restarted during testing and every task, project, client and invoice was still served afterwards (Test 6 below). Money is stored as integer paise, never floating-point rupees, so totals never drift by rounding.

5. Implementation Workflow: the Complete Flow

Using "add a studio task" as the worked example of frontend → API → database → response → display:

1. **Form (React):** the task form inside a project card holds title, due date and notes in controlled-input state and validates locally (title required, date picked).
2. **Fetch:** `api.createTask` sends `POST /api/tasks` with a JSON body; all URLs live in one file, `frontend/src/api.js`.
3. **Routing (Express):** CORS and JSON-parsing middleware run first; `routes/taskRoutes.js` maps POST / to `createTask`.
4. **Validation (server, again):** `validateTask` checks the projectId is a positive integer, title is required and ≤ 140 characters, and dueDate is a real calendar date in YYYY-MM-DD form (2026-02-31 is rejected, not silently rolled over).
5. **Business rule:** the controller confirms the project exists (404 if not).

6. **Database:** `prisma.studioTask.create` runs a parameterised INSERT; PostgreSQL enforces the foreign key, NOT NULL and length constraints.
7. **Response:** 201 Created with the saved row as JSON.
8. **Display:** the app re-fetches clients, projects, tasks, invoices and stats in one `Promise.all`; state updates and the task appears in the list - the screen always re-renders from stored database truth, never from local optimism.

6. API Reference: Route per CRUD Operation

Method + URL	Purpose	Success	Failure cases
GET /api/health	server + database check	200	503 DB down
GET /api/tasks?projectId=&done=	list tasks (Read)	200	400 bad filter
POST /api/tasks	add task (Create)	201	400 validation, 404 unknown project
GET /api/tasks/:id	one task (Read)	200	404
PUT /api/tasks/:id	edit task (Update)	200	400, 404
PATCH /api/tasks/:id/toggle	toggle done (Update)	200	404
DELETE /api/tasks/:id	delete task (Delete)	200	404
GET/POST/PUT/DELETE /api/clients(/:id)	client CRUD + search	200/201	400, 404, 409 duplicate email, 409 has projects
GET/POST/PUT/DELETE /api/projects(/:id)	project CRUD + status PATCH	200/201	400, 404
GET/POST/DELETE /api/invoices(/:id)	invoice raise/list/delete + mark-paid / void	200/201	400, 404, 409
GET /api/stats	dashboard counts and money aggregates	200	503

Why validation runs on both client and server. The browser check is for the user: instant, under the exact field, no round trip. But the browser can be bypassed entirely - curl and Postman talk to the API directly - so the server validates everything again before touching the database, and PostgreSQL's constraints are the final line of defence. Client-side validation is convenience; server-side validation is security.

7. Setup and Run

```
Prerequisites: Node.js 22+, PostgreSQL 16+ (a database named kaisei_client_desk)

# terminal 1 - backend
cd backend
Copy-Item .env.example .env    # put your PostgreSQL password in DATABASE_URL
```

```
npm install
npx prisma generate            # generate the Prisma client
npx prisma migrate deploy      # create the tables from versioned migrations
npm run seed                   # demo data (optional; only on an empty database)
npm run dev                    # API on http://localhost:5000

# terminal 2 - frontend
cd frontend
npm install
npm run dev                    # app on http://localhost:5173
```

Check `http://localhost:5000/api/health` - it answers `{"status":"ok","database":"connected"}`.

8. Test Results

Executed against the running application (React UI + API + PostgreSQL) on Oct 26, 2026.

#	Test	Request / action	Expected	Actual
1	Create task	`POST /api/tasks` {"projectId":3,"title":"Design booking-slot calendar UI","dueDate":"2026-11-05"}	201, new task JSON	201, task id 1 returned with all fields
2	Invalid input	`POST /api/tasks` {"projectId":3,"title":"","dueDate":"2026-11-05"}	400 error	400 {"error":"Validation failed","details": {"title":"Title is required"}}
3	Missing ID	`DELETE /api/tasks/99`	404 not found	404 {"error":"Record not found"}
4	Toggle done (Update)	`PATCH /api/tasks/5/toggle` and the UI checkbox	done flips false→true; row shows struck-through	done: true in JSON; UI row struck through (Figure 3)
5	Business conflict	duplicate client email; pay an already-paid invoice; delete a client with projects	409 with a readable message	409 "A client with this email already exists"; 409 "Invoice 1 is already paid"; 409 "still has 2 project(s)"
6	Persistence across restart	stop the backend, start it again, `GET /api/tasks`	all tasks still present	all 5 tasks returned after restart
7	Seed recreation	fresh empty database: `prisma migrate deploy` + `npm run seed`	demo dataset recreated	4 clients, 5 projects, 4 invoices, 5 tasks created and counted

9. Screenshots

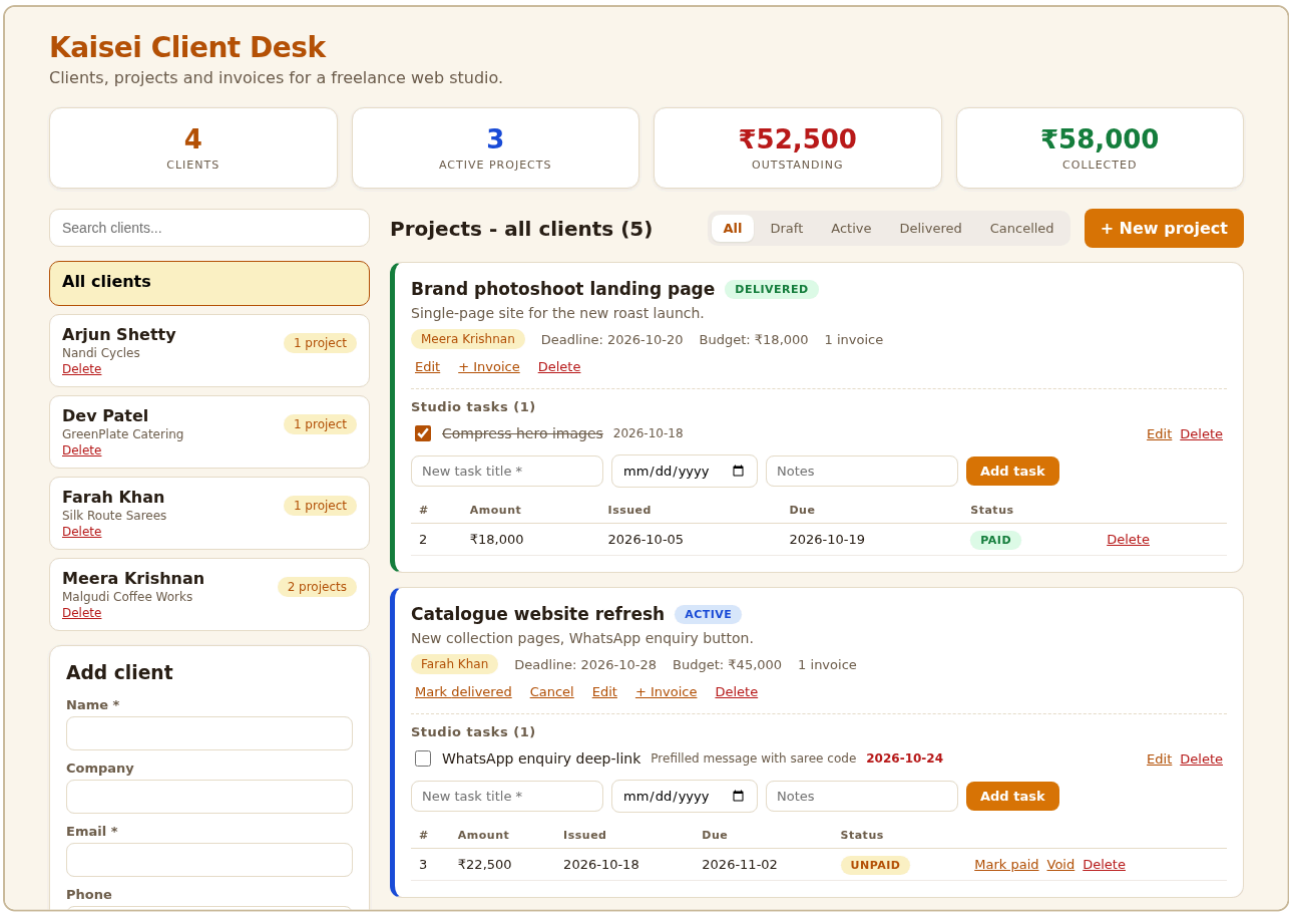


Figure 1 - Read: the task list inside each project card, with the dashboard aggregates and the invoice tables.

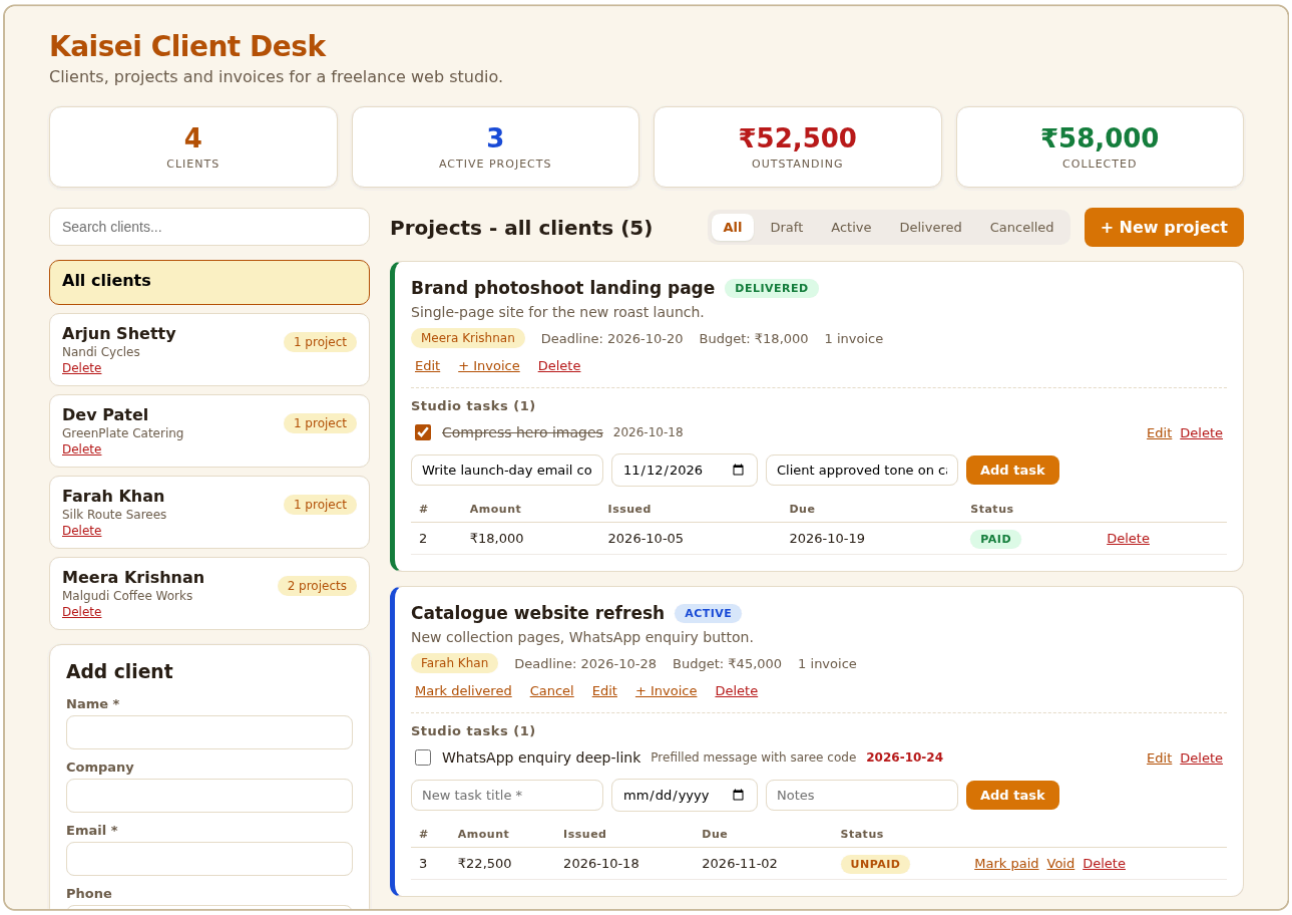


Figure 2 - Create: the add-task form filled in ("Write launch-day email copy", due 2026-11-12).

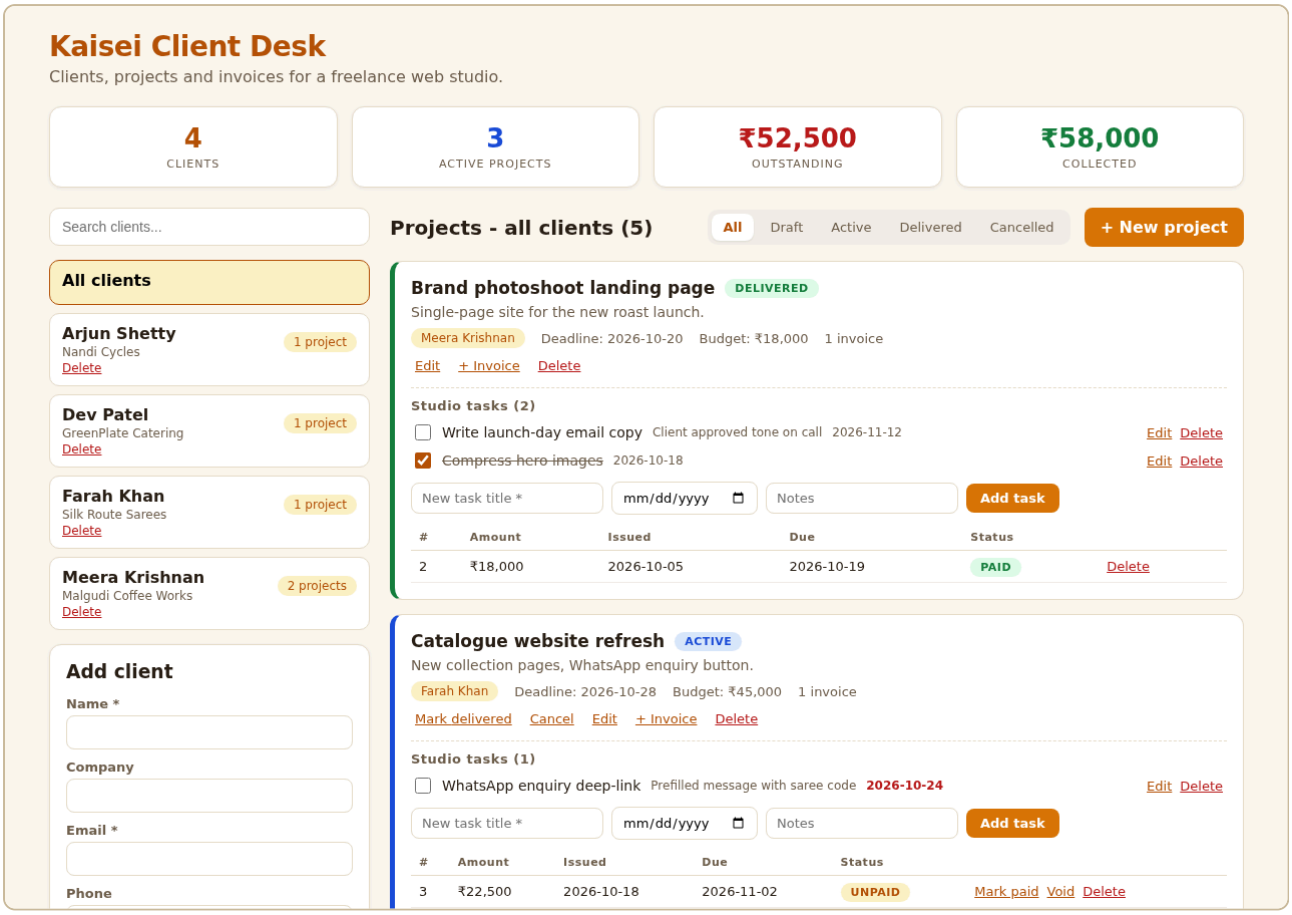


Figure 3 - Create result: the new task stored and rendered in the list after the 201 response and re-fetch.

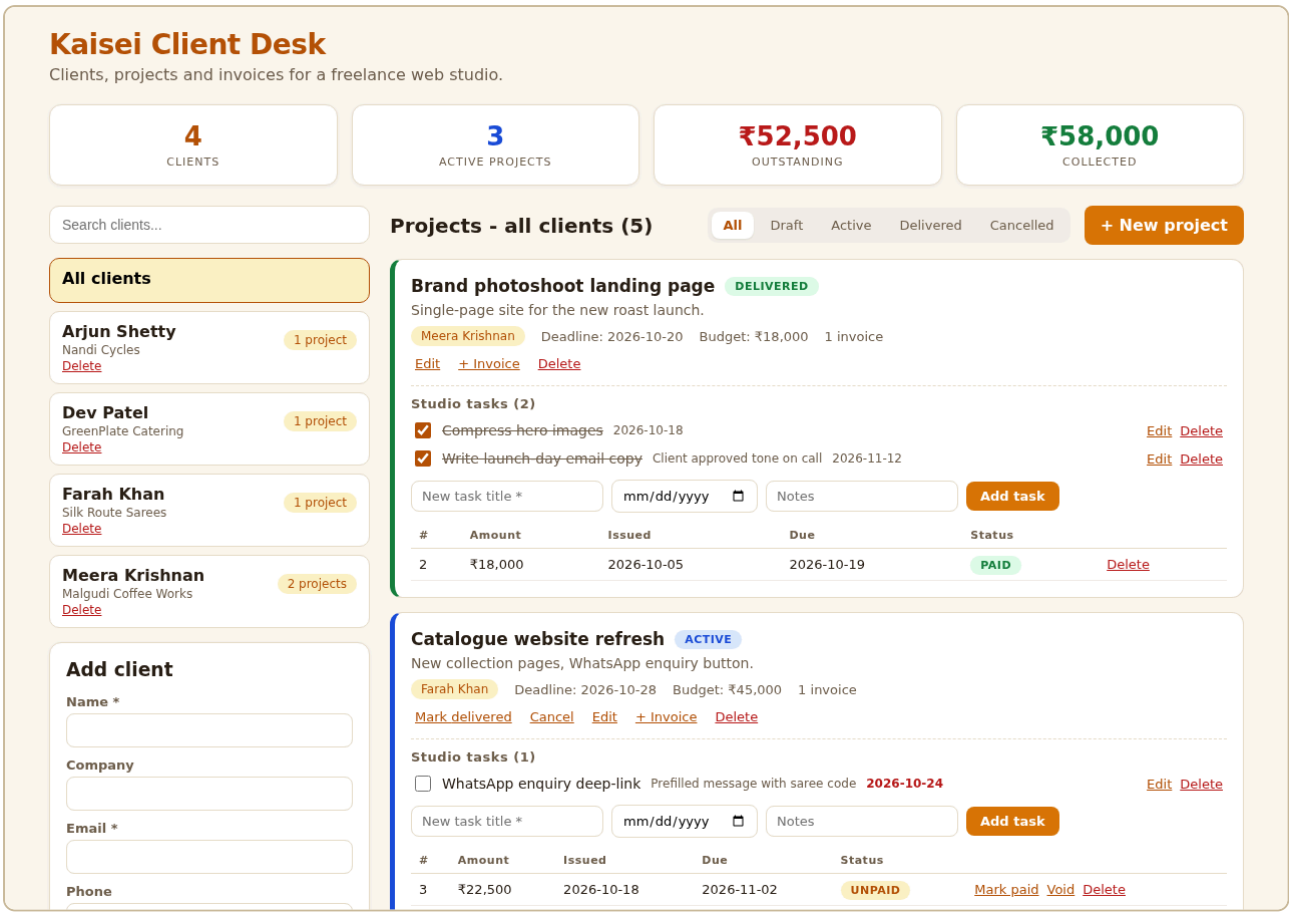


Figure 4 - Update: the same task toggled done - checkbox checked, title struck through.

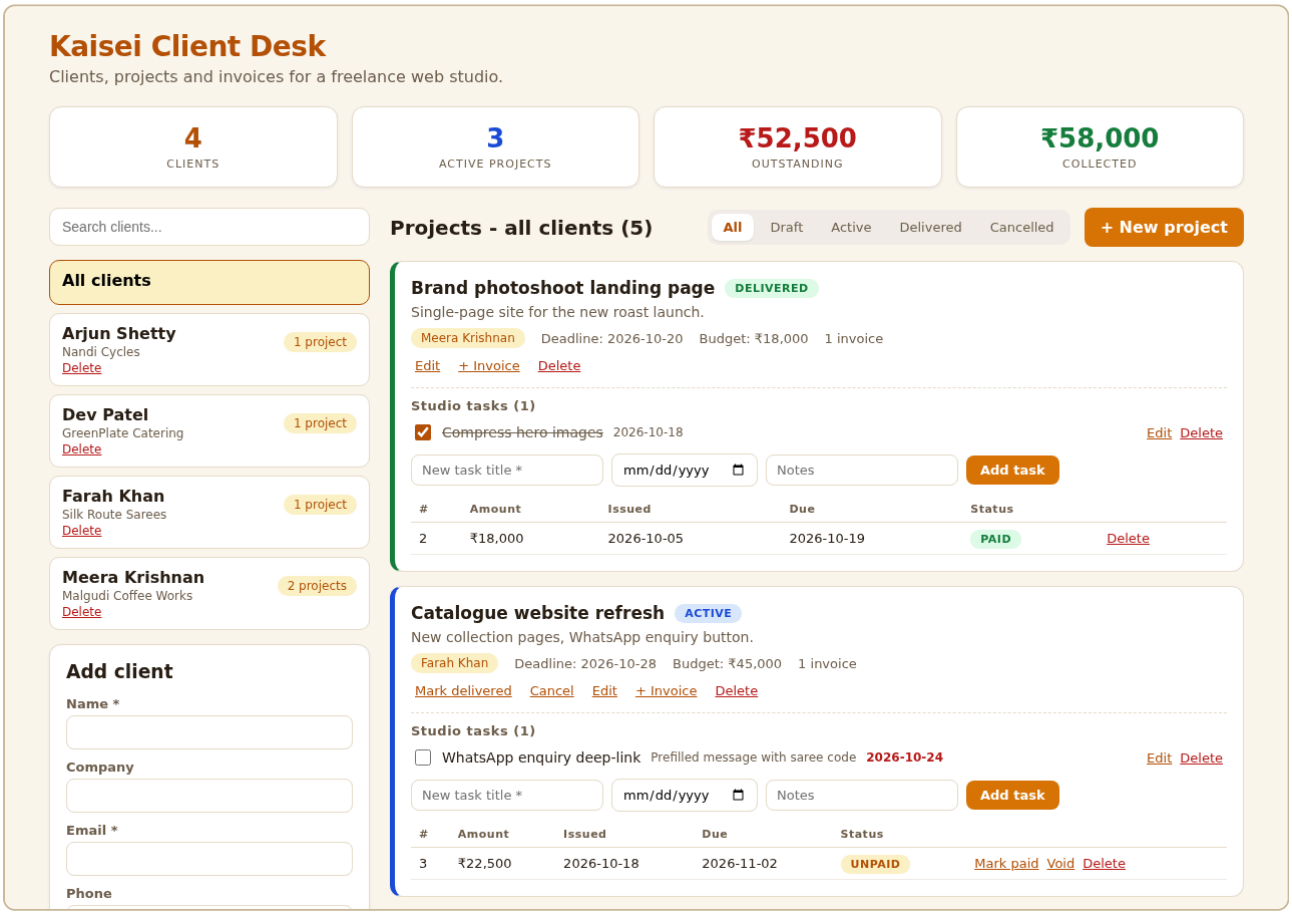


Figure 5 - Delete: the list after deleting that task - it is gone from the database and the screen.

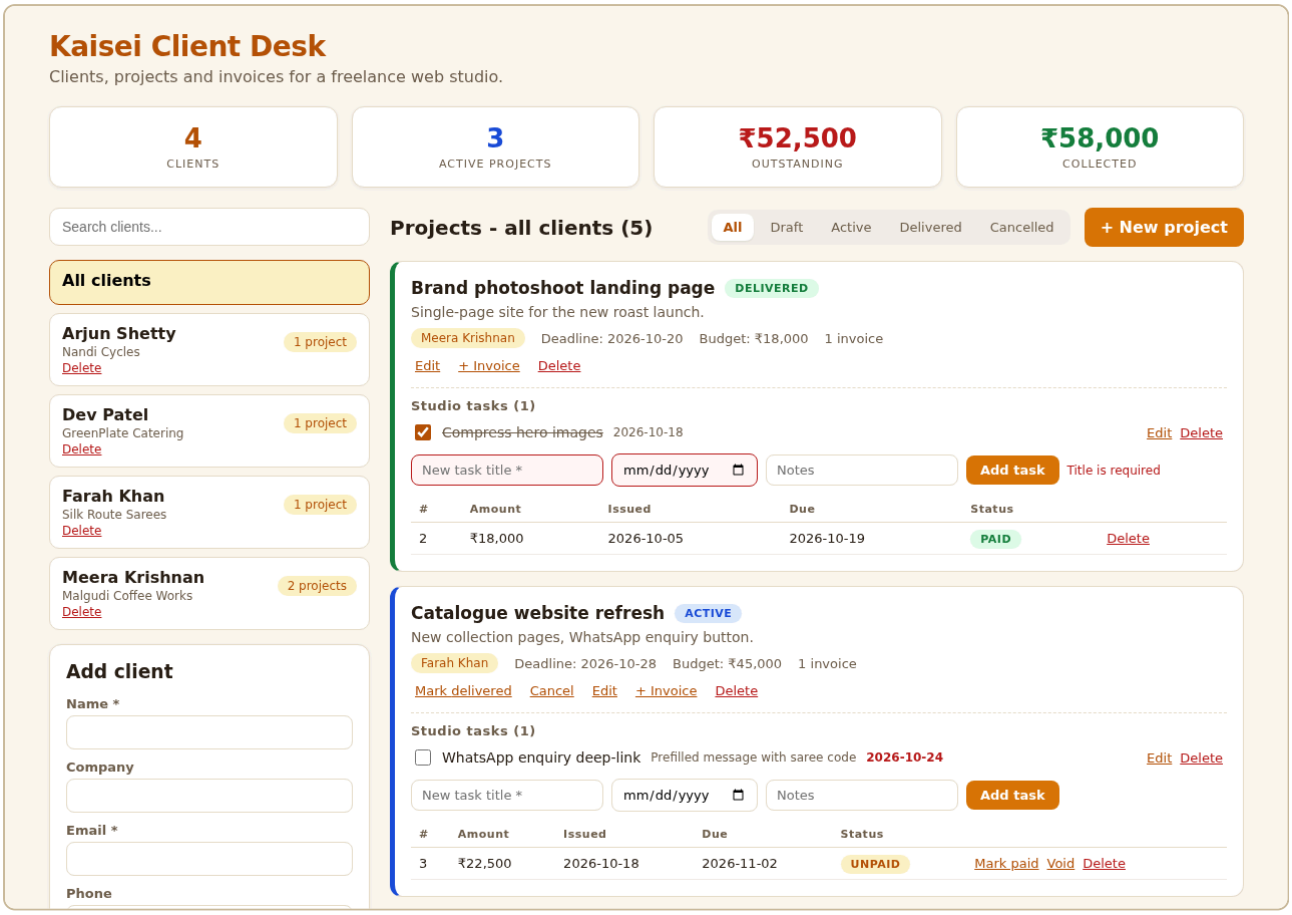


Figure 6 - Validation: submitting the empty task form is blocked with "Title is required"; the same rule fires server-side as a 400 (Test 2).

10. Conclusion

Kaisei Client Desk demonstrates the full path the assignment asks for: a browser interface where users create, view, update and delete tasks (and the clients, projects and invoices around them); an Express REST API that validates every input and answers with honest status codes; and a PostgreSQL database whose constraints, foreign keys and migrations keep the data correct and permanent. The same layered structure - routes, controllers, validation, ORM, database - would carry login, pagination or deployment without redesign.